Flask

一、快速开始

1. 安装

```
pip install flask
```

2. 最小示例

```
from flask import Flask, jsonify, abort

app = Flask(__name__)

@app.route("/")
def hello():
    return jsonify({"message": "Hello World!"})

if __name__ == "__main__":
    app.run(debug=True)
```

3. 项目结构

```
project/
├── app/
│   ├── __init__.py          # 应用工厂
│   ├── extensions.py        # 扩展初始化
│   ├── config.py            # 配置类
│   ├── models/              # 数据模型
│   ├── routes/              # 路由蓝图
│   ├── services/            # 业务逻辑
│   ├── schemas/             # 数据验证
│   └── utils/               # 工具函数
├── tests/
├── migrations/
├── requirements.txt
├── .env                     # 环境变量
└── run.py
```

4. 配置管理

```
import os
from dotenv import load_dotenv

load_dotenv()  # 加载.env文件

class Config:
    SECRET_KEY = os.environ.get('SECRET_KEY', 'dev-key')
    SQLALCHEMY_DATABASE_URI = os.environ.get('DATABASE_URL',
'sqlite:///app.db')
    SQLALCHEMY_TRACK_MODIFICATIONS = False

class DevelopmentConfig(Config):
    DEBUG = True

class ProductionConfig(Config):
    DEBUG = False

class TestingConfig(Config):
    TESTING = True
    SQLALCHEMY_DATABASE_URI = 'sqlite:///:memory:'

config = {
    'development': DevelopmentConfig,
    'production': ProductionConfig,
    'testing': TestingConfig
}
```

环境变量文件 `.env`

```
SECRET_KEY=your-secret-key
DATABASE_URL=postgresql://user:pass@localhost/db
REDIS_URL=redis://localhost:6379/0
```

二、路由与请求

1. 路由

基本路由

```
@app.route('/users')
def get_users():
    return jsonify({'users': []})
```

```
@app.route('/users/<int:user_id>')
def get_user(user_id):
    return jsonify({'id': user_id})
```

路由转换器

转换器	说明
`string`	默认，不含斜杠的文本
`int`	正整数
`float`	正浮点数
`path`	含斜杠的文本
`uuid`	UUID字符串

HTTP方法

```
@app.route('/users', methods=['GET', 'POST'])
def users():
    if request.method == 'POST':
        return create_user()
    return get_users()

# 便捷方法
@app.get('/users')
def list_users():
    return jsonify({'users': []})

@app.post('/users')
def create_user():
    return jsonify({'id': 1}), 201

@app.put('/users/<int:user_id>')
def update_user(user_id):
    return jsonify({'id': user_id})

@app.delete('/users/<int:user_id>')
def delete_user(user_id):
    return '', 204
```

URL构建

```
from flask import url_for, request

url_for('get_user', user_id=1)  # '/users/1'
url_for('list_users', page=2)   # '/users?page=2'
```

2. 请求对象

```python
from flask import request

@app.route('/api/data', methods=['POST'])
def handle_data():
    # JSON数据
    json_data = request.get_json()

    # 查询参数
    page = request.args.get('page', 1, type=int)
    size = request.args.get('size', 10, type=int)

    # 表单数据
    name = request.form.get('name')

    # 文件上传
    file = request.files.get('file')

    # 请求头
    auth = request.headers.get('Authorization')
    content_type = request.content_type

    # Cookie
    session_id = request.cookies.get('session_id')

    # 客户端IP
    ip = request.remote_addr

    # 请求路径
    path = request.path
    endpoint = request.endpoint

    return jsonify({'received': True})
```

3. 响应对象

```python
from flask import jsonify, make_response, Response

# 返回JSON
@app.route('/api/user')
def get_user():
    return jsonify({'id': 1, 'name': '张三'})

# 返回元组 (响应体, 状态码, 响应头)
@app.route('/api/create')
def create():
    return {'id': 1}, 201, {'Location': '/api/users/1'}

# 自定义响应
```

```python
@app.route('/api/download')
def download():
    resp = make_response('file content')
    resp.headers['Content-Type'] = 'text/plain'
    resp.headers['Content-Disposition'] = 'attachment; filename=data.txt'
    return resp

# 设置Cookie
@app.route('/api/set-cookie')
def set_cookie():
    resp = jsonify({'status': 'ok'})
    resp.set_cookie('token', 'abc123', max_age=3600, httponly=True,
secure=True)
    return resp

# 流式响应
@app.route('/api/stream')
def stream():
    def generate():
        for i in range(100):
            yield f"data: {i}\n"
    return Response(generate(), mimetype='text/plain')
```

三、蓝图与模块化

1. 基本使用

```python
# routes/user.py
from flask import Blueprint, jsonify, request

user_bp = Blueprint('user', __name__, url_prefix='/api/users')

@user_bp.route('/')
def list_users():
    return jsonify({'users': []})

@user_bp.route('/<int:user_id>')
def get_user(user_id):
    return jsonify({'id': user_id})

@user_bp.route('/', methods=['POST'])
def create_user():
    data = request.get_json()
    return jsonify(data), 201

# app/__init__.py
from flask import Flask
from routes.user import user_bp

def create_app():
```

```
    app = Flask(__name__)
    app.register_blueprint(user_bp)
    return app
```

2. API版本控制

```
# v1版本
api_v1 = Blueprint('api_v1', __name__, url_prefix='/api/v1')

@api_v1.route('/users')
def list_users_v1():
    return jsonify({'version': 1, 'users': []})

# v2版本
api_v2 = Blueprint('api_v2', __name__, url_prefix='/api/v2')

@api_v2.route('/users')
def list_users_v2():
    return jsonify({'version': 2, 'users': [], 'total': 0})

# 注册
app.register_blueprint(api_v1)
app.register_blueprint(api_v2)
```

四、请求验证

1. Marshmallow

```
pip install marshmallow
```

```
from marshmallow import Schema, fields, validate, ValidationError,
post_load
from flask import request, jsonify, abort

class UserSchema(Schema):
    id = fields.Integer(dump_only=True)
    username = fields.String(required=True, validate=validate.Length(min=3,
max=30))
    email = fields.Email(required=True)
    password = fields.String(required=True, load_only=True)
    age = fields.Integer(validate=validate.Range(min=0, max=150))
    created_at = fields.DateTime(dump_only=True)

    @post_load
    def make_user(self, data, **kwargs):
        # 创建对象
```

```
        return User(**data)

user_schema = UserSchema()
users_schema = UserSchema(many=True)

@app.route('/api/users', methods=['POST'])
def create_user():
    try:
        data = user_schema.load(request.get_json())
    except ValidationError as err:
        return jsonify({'errors': err.messages}), 400

    db.session.add(data)
    db.session.commit()
    return jsonify(user_schema.dump(data)), 201

@app.route('/api/users')
def list_users():
    users = User.query.all()
    return jsonify(users_schema.dump(users))

# 部分更新
class UserUpdateSchema(Schema):
    username = fields.String(validate=validate.Length(min=3, max=30))
    email = fields.Email()
    age = fields.Integer(validate=validate.Range(min=0, max=150))

user_update_schema = UserUpdateSchema()

@app.route('/api/users/<int:user_id>', methods=['PATCH'])
def update_user(user_id):
    user = db.session.get(User, user_id) or abort(404)
    try:
        data = user_update_schema.load(request.get_json(), partial=True)
    except ValidationError as err:
        return jsonify({'errors': err.messages}), 400

    for key, value in data.items():
        setattr(user, key, value)
    db.session.commit()
    return jsonify(user_schema.dump(user))
```

2. Pydantic

```
pip install pydantic[email]
```

```
from pydantic import BaseModel, EmailStr, field_validator, Field
from typing import Optional
from datetime import datetime
```

```
class UserCreate(BaseModel):
    username: str = Field(..., min_length=3, max_length=30)
    email: EmailStr
    password: str = Field(..., min_length=6)
    age: Optional[int] = Field(None, ge=0, le=150)

    @field_validator('username')
    @classmethod
    def validate_username(cls, v):
        if not v.isalnum():
            raise ValueError('必须为字母数字')
        return v

class UserResponse(BaseModel):
    id: int
    username: str
    email: str
    age: Optional[int]
    created_at: datetime

    class Config:
        from_attributes = True

@app.route('/api/users', methods=['POST'])
def create_user():
    try:
        data = UserCreate(**request.get_json())
    except ValueError as e:
        return jsonify({'error': str(e)}), 400

    user = User(**data.model_dump())
    db.session.add(user)
    db.session.commit()
    return jsonify(UserResponse.model_validate(user).model_dump()), 201
```

五、数据库集成

1. Flask-SQLAlchemy

```
pip install flask-sqlalchemy
```

```
from flask import Flask
from flask_sqlalchemy import SQLAlchemy
from datetime import datetime, timezone

app = Flask(__name__)
app.config['SQLALCHEMY_DATABASE_URI'] =
```

```
'postgresql://user:pass@localhost/db'
db = SQLAlchemy(app)

class User(db.Model):
    __tablename__ = 'users'

    id = db.Column(db.Integer, primary_key=True)
    username = db.Column(db.String(80), unique=True, nullable=False)
    email = db.Column(db.String(120), unique=True)
    password_hash = db.Column(db.String(256))
    is_active = db.Column(db.Boolean, default=True)
    created_at = db.Column(db.DateTime, default=lambda:
datetime.now(timezone.utc))
    updated_at = db.Column(db.DateTime, default=lambda:
datetime.now(timezone.utc), onupdate=lambda: datetime.now(timezone.utc))

    # 关系
    posts = db.relationship('Post', backref='author', lazy='dynamic')

    def to_dict(self):
        return {
            'id': self.id,
            'username': self.username,
            'email': self.email,
            'is_active': self.is_active,
            'created_at': self.created_at.isoformat() if self.created_at
else None
        }

class Post(db.Model):
    id = db.Column(db.Integer, primary_key=True)
    title = db.Column(db.String(200), nullable=False)
    content = db.Column(db.Text)
    user_id = db.Column(db.Integer, db.ForeignKey('users.id'))
    created_at = db.Column(db.DateTime, default=lambda:
datetime.now(timezone.utc))
```

2. CRUD操作

```
from flask import request, abort

# 创建
@app.route('/api/users', methods=['POST'])
def create_user():
    data = request.get_json()
    user = User(username=data['username'], email=data['email'])
    db.session.add(user)
    db.session.commit()
    return jsonify(user.to_dict()), 201

# 查询列表
```

```python
@app.route('/api/users')
def list_users():
    page = request.args.get('page', 1, type=int)
    size = request.args.get('size', 10, type=int)

    pagination = User.query.filter_by(is_active=True)\
        .order_by(User.created_at.desc())\
        .paginate(page=page, per_page=size, error_out=False)

    return jsonify({
        'items': [u.to_dict() for u in pagination.items],
        'total': pagination.total,
        'pages': pagination.pages,
        'current_page': page
    })

# 查询单个
@app.route('/api/users/<int:user_id>')
def get_user(user_id):
    user = db.session.get(User, user_id) or abort(404)
    return jsonify(user.to_dict())

# 更新
@app.route('/api/users/<int:user_id>', methods=['PUT'])
def update_user(user_id):
    user = db.session.get(User, user_id) or abort(404)
    data = request.get_json()
    user.email = data.get('email', user.email)
    db.session.commit()
    return jsonify(user.to_dict())

# 删除
@app.route('/api/users/<int:user_id>', methods=['DELETE'])
def delete_user(user_id):
    user = db.session.get(User, user_id) or abort(404)
    db.session.delete(user)
    db.session.commit()
    return '', 204

# 软删除
@app.route('/api/users/<int:user_id>', methods=['DELETE'])
def soft_delete_user(user_id):
    user = db.session.get(User, user_id) or abort(404)
    user.is_active = False
    db.session.commit()
    return '', 204
```

3. 数据库迁移

```
pip install flask-migrate
```

```
flask db init                     # 初始化
flask db migrate -m "initial"     # 生成迁移
flask db upgrade                  # 执行迁移
flask db downgrade                # 回滚
```

六、认证与授权

1. JWT认证

```
pip install flask-jwt-extended
```

```
from flask import Flask, jsonify, request
from flask_jwt_extended import (
    JWTManager, create_access_token, create_refresh_token,
    get_jwt_identity, jwt_required, get_jwt
)
from datetime import timedelta, datetime, timezone
from werkzeug.security import check_password_hash

app = Flask(__name__)
app.config['JWT_SECRET_KEY'] = 'your-secret-key'
app.config['JWT_ACCESS_TOKEN_EXPIRES'] = timedelta(hours=1)
app.config['JWT_REFRESH_TOKEN_EXPIRES'] = timedelta(days=30)
jwt = JWTManager(app)

@app.route('/login', methods=['POST'])
def login():
    data = request.get_json()
    username = data.get('username')
    password = data.get('password')

    user = User.query.filter_by(username=username).first()
    if user and check_password_hash(user.password_hash, password):
        access_token = create_access_token(identity=user.id)
        refresh_token = create_refresh_token(identity=user.id)
        return jsonify({
            'access_token': access_token,
            'refresh_token': refresh_token
        })
    return jsonify({'error': 'Invalid credentials'}), 401

@app.route('/refresh', methods=['POST'])
@jwt_required(refresh=True)
def refresh():
    user_id = get_jwt_identity()
    access_token = create_access_token(identity=user_id)
    return jsonify({'access_token': access_token})
```

```python
@app.route('/protected')
@jwt_required()
def protected():
    user_id = get_jwt_identity()
    claims = get_jwt()
    return jsonify({'user_id': user_id, 'role': claims.get('role')})

# 角色验证
from functools import wraps

def role_required(role):
    def decorator(f):
        @wraps(f)
        @jwt_required()
        def decorated_function(*args, **kwargs):
            claims = get_jwt()
            if claims.get('role') != role:
                return jsonify({'error': 'Insufficient permissions'}), 403
            return f(*args, **kwargs)
        return decorated_function
    return decorator

@app.route('/admin')
@role_required('admin')
def admin_only():
    return jsonify({'message': 'Admin access'})
```

2. API Key认证

```python
from functools import wraps

def require_api_key(f):
    @wraps(f)
    def decorated(*args, **kwargs):
        api_key = request.headers.get('X-API-Key')
        if not api_key:
            return jsonify({'error': 'API key required'}), 401

        key = ApiKey.query.filter_by(key=api_key, is_active=True).first()
        if not key:
            return jsonify({'error': 'Invalid API key'}), 401

        # 记录使用
        key.last_used = datetime.now(timezone.utc)
        db.session.commit()

        return f(*args, **kwargs)
    return decorated

@app.route('/api/protected')
@require_api_key
```

```
def protected():
    return jsonify({'data': 'secret'})
```

3. Basic认证

```
from functools import wraps

def basic_auth_required(f):
    @wraps(f)
    def decorated(*args, **kwargs):
        auth = request.authorization
        if not auth or not verify_user(auth.username, auth.password):
            return jsonify({'error': 'Unauthorized'}), 401, {
                'WWW-Authenticate': 'Basic realm="Login Required"'
            }
        return f(*args, **kwargs)
    return decorated
```

七、文件处理

1. 文件上传

```
from werkzeug.utils import secure_filename
from datetime import datetime, timezone
import os

ALLOWED_EXTENSIONS = {'txt', 'pdf', 'png', 'jpg', 'jpeg', 'gif'}
UPLOAD_FOLDER = 'uploads'

app.config['UPLOAD_FOLDER'] = UPLOAD_FOLDER
app.config['MAX_CONTENT_LENGTH'] = 16 * 1024 * 1024  # 16MB

def allowed_file(filename):
    return '.' in filename and \
           filename.rsplit('.', 1)[1].lower() in ALLOWED_EXTENSIONS

@app.route('/api/upload', methods=['POST'])
def upload_file():
    if 'file' not in request.files:
        return jsonify({'error': 'No file'}), 400

    file = request.files['file']
    if file.filename == '':
        return jsonify({'error': 'No selected file'}), 400

    if file and allowed_file(file.filename):
        filename = secure_filename(file.filename)
        # 添加时间戳避免重名
```

```
        filename = f"
{datetime.now(timezone.utc).strftime('%Y%m%d%H%M%S')}_{filename}"
        filepath = os.path.join(app.config['UPLOAD_FOLDER'], filename)
        file.save(filepath)

        return jsonify({
            'filename': filename,
            'url': f'/uploads/{filename}'
        })
    return jsonify({'error': 'File type not allowed'}), 400

# 多文件上传
@app.route('/api/upload/multiple', methods=['POST'])
def upload_multiple():
    files = request.files.getlist('files')
    results = []
    for file in files:
        if file and allowed_file(file.filename):
            filename = secure_filename(file.filename)
            file.save(os.path.join(app.config['UPLOAD_FOLDER'], filename))
            results.append({'filename': filename})
    return jsonify({'files': results})
```

2. 文件下载

```
from flask import send_file, send_from_directory, jsonify
import os

@app.route('/api/download/<filename>')
def download_file(filename):
    try:
        return send_from_directory(
            app.config['UPLOAD_FOLDER'],
            filename,
            as_attachment=True
        )
    except FileNotFoundError:
        return jsonify({'error': 'File not found'}), 404

# 流式下载大文件
@app.route('/api/download/large/<filename>')
def download_large(filename):
    def generate():
        with open(os.path.join(app.config['UPLOAD_FOLDER'], filename),
'rb') as f:
            while chunk := f.read(8192):
                yield chunk
    return Response(generate(), mimetype='application/octet-stream')
```

八、错误处理

```
from flask import jsonify
from werkzeug.exceptions import HTTPException

class APIError(Exception):
    def __init__(self, message, status_code=400, payload=None):
        self.message = message
        self.status_code = status_code
        self.payload = payload

    def to_dict(self):
        rv = {'error': self.message}
        if self.payload:
            rv.update(self.payload)
        return rv

class ResourceNotFound(APIError):
    def __init__(self, resource, resource_id):
        super().__init__(
            f'{resource} not found',
            status_code=404,
            payload={'resource': resource, 'id': resource_id}
        )

class ValidationError(APIError):
    def __init__(self, message, errors=None):
        super().__init__(message, status_code=422, payload={'errors':
errors})

@app.errorhandler(APIError)
def handle_api_error(error):
    return jsonify(error.to_dict()), error.status_code

@app.errorhandler(404)
def not_found(error):
    return jsonify({'error': 'Resource not found'}), 404

@app.errorhandler(500)
def internal_error(error):
    db.session.rollback()
    app.logger.error(f'Internal error: {error}')
    return jsonify({'error': 'Internal server error'}), 500

@app.errorhandler(Exception)
def handle_exception(e):
    if isinstance(e, HTTPException):
        return jsonify({'error': e.description}), e.code
    app.logger.exception('Unhandled exception')
    return jsonify({'error': 'Internal server error'}), 500

# 使用
```

```
@app.route('/api/users/<int:user_id>')
def get_user(user_id):
    user = db.session.get(User, user_id)
    if not user:
        raise ResourceNotFound('User', user_id)
    return jsonify(user.to_dict())
```

九、缓存策略

1. Flask-Caching

```
pip install flask-caching
```

```
from flask import request, abort
from flask_caching import Cache

app.config['CACHE_TYPE'] = 'RedisCache'
app.config['CACHE_REDIS_URL'] = 'redis://localhost:6379/0'
app.config['CACHE_DEFAULT_TIMEOUT'] = 300

cache = Cache(app)

# 视图缓存
@app.route('/api/expensive')
@cache.cached(timeout=60)
def expensive_operation():
    return jsonify({'result': compute()})

# 按参数缓存
@app.route('/api/users/<int:user_id>')
@cache.cached(timeout=300, key_prefix=lambda:
f"user_{request.view_args['user_id']}")
def get_user(user_id):
    user = db.session.get(User, user_id) or abort(404)
    return jsonify(user.to_dict())

# 函数缓存（使用cached装饰器）
def get_user_by_id(user_id):
    return db.session.get(User, user_id)

cached_get_user = cache.cached(timeout=300, key_prefix='user')
(get_user_by_id)

# 清除缓存
@app.route('/api/users/<int:user_id>', methods=['PUT'])
def update_user(user_id):
    user = db.session.get(User, user_id) or abort(404)
    # ... 更新
```

```
    cache.delete(f"user_{user_id}")
    cache.delete(f"user")  # 删除cached包装后的缓存
    return jsonify(user.to_dict())
```

2. Redis直接使用

```
import redis
import json

redis_client = redis.Redis(host='localhost', port=6379, db=0,
decode_responses=True)

@app.route('/api/products')
def get_products():
    cache_key = 'products:all'

    # 尝试从缓存获取
    cached = redis_client.get(cache_key)
    if cached:
        return jsonify(json.loads(cached))

    # 数据库查询
    products = [p.to_dict() for p in Product.query.all()]

    # 存入缓存
    redis_client.setex(cache_key, 300, json.dumps(products))

    return jsonify(products)

# 缓存更新
def invalidate_product_cache():
    redis_client.delete('products:all')
```

十、限流与防护

1. Flask-Limiter

```
pip install flask-limiter
```

```
from flask_limiter import Limiter
from flask_limiter.util import get_remote_address
from flask_limiter.errors import RateLimitExceeded

limiter = Limiter(
    app,
    key_func=get_remote_address,
```

```
    storage_uri='redis://localhost:6379',
    default_limits=['200 per day', '50 per hour']
)

# 视图限流
@app.route('/api/data')
@limiter.limit('10 per minute')
def get_data():
    return jsonify({'data': '...'})

# 多重限制
@app.route('/api/login', methods=['POST'])
@limiter.limit('5 per minute')
@limiter.limit('20 per hour')
def login():
    return jsonify({'token': '...'})

# 按用户限流
def get_user_key():
    if hasattr(g, 'user_id'):
        return f"user:{g.user_id}"
    return f"ip:{get_remote_address()}"

limiter = Limiter(app, key_func=get_user_key)

# 白名单
import ipaddress

@limiter.request_filter
def ip_whitelist():
    whitelist = ['127.0.0.1']
    whitelist_networks = [ipaddress.ip_network('192.168.1.0/24')]

    if request.remote_addr in whitelist:
        return True

    try:
        ip = ipaddress.ip_address(request.remote_addr)
        return any(ip in network for network in whitelist_networks)
    except ValueError:
        return False

# 错误处理
@app.errorhandler(RateLimitExceeded)
def ratelimit_handler(e):
    return jsonify({
        'error': 'Rate limit exceeded',
        'message': str(e.description)
    }), 429
```

2. 熔断器

```python
import time
import threading
from functools import wraps

class CircuitBreaker:
    def __init__(self, threshold=5, timeout=60,
expected_exception=Exception):
        self.threshold = threshold
        self.timeout = timeout
        self.expected_exception = expected_exception
        self.failures = 0
        self.state = 'closed'
        self.last_failure = 0
        self.lock = threading.Lock()

    def __call__(self, func):
        @wraps(func)
        def wrapper(*args, **kwargs):
            with self.lock:
                if self.state == 'open':
                    if time.time() - self.last_failure > self.timeout:
                        self.state = 'half-open'
                    else:
                        raise Exception('Circuit breaker is open')

            try:
                result = func(*args, **kwargs)
                with self.lock:
                    if self.state == 'half-open':
                        self.state = 'closed'
                        self.failures = 0
                return result
            except self.expected_exception as e:
                with self.lock:
                    self.failures += 1
                    self.last_failure = time.time()
                    if self.failures >= self.threshold:
                        self.state = 'open'
                raise
        return wrapper

breaker = CircuitBreaker(threshold=3, timeout=30)

@app.route('/api/external')
@breaker
def call_external():
    return requests.get('http://external-api/data', timeout=5).json()
```

十一、异步任务

Celery

```
pip install celery redis
```

```
# tasks.py
from celery import Celery
from celery.schedules import crontab

celery = Celery('tasks')
celery.config_from_object({
    'broker_url': 'redis://localhost:6379/0',
    'result_backend': 'redis://localhost:6379/1',
    'task_serializer': 'json',
    'result_serializer': 'json',
    'timezone': 'Asia/Shanghai',
    'beat_schedule': {
        'cleanup-every-night': {
            'task': 'tasks.cleanup_expired_sessions',
            'schedule': crontab(hour=3, minute=0),
        },
    }
})

@celery.task
def send_email(to, subject, body):
    # 发送邮件逻辑
    return {'sent': to}

@celery.task(bind=True, max_retries=3, default_retry_delay=60)
def process_file(self, file_path):
    try:
        return do_process(file_path)
    except Exception as exc:
        raise self.retry(exc=exc)

@celery.task
def generate_report(user_id):
    # 生成报表
    return {'report_url': '...'}

# app.py
@app.route('/api/send-email', methods=['POST'])
def trigger_email():
    data = request.get_json()
    task = send_email.delay(data['to'], data['subject'], data['body'])
    return jsonify({'task_id': task.id})

@app.route('/api/tasks/<task_id>')
def task_status(task_id):
    task = celery.AsyncResult(task_id)
```

```
    return jsonify({
        'status': task.status,
        'result': task.result if task.ready() else None
    })

# 链式任务
@app.route('/api/process')
def process():
    from celery import chain
    result = chain(
        process_file.s('file1.txt'),
        generate_report.s()
    ).apply_async()
    return jsonify({'chain_id': result.id})
```

启动Worker

```
celery -A tasks worker --loglevel=info
celery -A tasks beat --loglevel=info  # 定时任务调度器
```

十二、实时通信

1. SSE (Server-Sent Events)

```
from flask import Response
import json
import time
import queue

# 简单SSE
@app.route('/api/stream')
def stream():
    def generate():
        for i in range(100):
            yield f"data: {json.dumps({'count': i, 'time':
time.time()})}\n\n"
            time.sleep(1)
    return Response(
        generate(),
        mimetype='text/event-stream',
        headers={
            'Cache-Control': 'no-cache',
            'X-Accel-Buffering': 'no'  # 禁用Nginx缓冲
        }
    )

# 带事件类型
@app.route('/api/events')
```

```
def events():
    def event_stream():
        while True:
            data = get_event_data()
            yield f"event: message\ndata: {json.dumps(data)}\n\n"
            time.sleep(1)
    return Response(event_stream(), mimetype='text/event-stream')

# 广播消息
clients = []

@app.route('/api/subscribe')
def subscribe():
    def stream():
        q = queue.Queue()
        clients.append(q)
        try:
            while True:
                data = q.get()
                yield f"data: {json.dumps(data)}\n\n"
        finally:
            clients.remove(q)
    return Response(stream(), mimetype='text/event-stream')

def broadcast(message):
    for q in clients:
        q.put(message)
```

2. WebSocket

```
pip install flask-socketio
```

```
from flask_socketio import SocketIO, emit, join_room, leave_room, rooms

socketio = SocketIO(app, cors_allowed_origins="*",
message_queue='redis://localhost:6379/0')

@socketio.on('connect')
def handle_connect():
    user_id = get_current_user_id()
    join_room(f'user_{user_id}')
    emit('connected', {'message': 'Welcome'})

@socketio.on('disconnect')
def handle_disconnect():
    emit('disconnected', broadcast=True)

@socketio.on('message')
def handle_message(data):
```

```
    emit('response', data, room=data.get('room'))

@socketio.on('join')
def on_join(data):
    join_room(data['room'])
    emit('status', {'msg': f'User joined {data["room"]}'},
room=data['room'])

@socketio.on('leave')
def on_leave(data):
    leave_room(data['room'])
    emit('status', {'msg': f'User left'}, room=data['room'])

# 广播
def notify_user(user_id, message):
    socketio.emit('notification', message, room=f'user_{user_id}')

if __name__ == '__main__':
    socketio.run(app, debug=True)
```

十三、中间件

```
from flask import g, request
from werkzeug.routing import BuildError
import time
import uuid

# 请求前
@app.before_request
def before_request():
    g.start_time = time.time()
    g.request_id = request.headers.get('X-Request-ID', str(uuid.uuid4()))

# 请求后
@app.after_request
def after_request(response):
    # 添加响应头
    response.headers['X-Request-ID'] = g.get('request_id', '')
    response.headers['X-Response-Time'] = f"{time.time() -
g.start_time:.3f}s"
    return response

# 请求结束
@app.teardown_request
def teardown_request(exception=None):
    if exception:
        app.logger.error(f'Request failed: {exception}')
    db.session.remove()

# 首次请求（Flask 3.0已移除before_first_request，改用其他方式）
```

```
# 方式1: 在create_app中执行
# 方式2: 使用扩展初始化时执行

# CORS
from flask_cors import CORS
CORS(app, resources={
    r"/api/*": {
        "origins": ["https://example.com"],
        "methods": ["GET", "POST", "PUT", "DELETE"],
        "allow_headers": ["Content-Type", "Authorization"]
    }
})

# ProxyFix
from werkzeug.middleware.proxy_fix import ProxyFix
app.wsgi_app = ProxyFix(app.wsgi_app, x_for=1, x_proto=1, x_host=1)
```

十四、健康检查与监控

```
from datetime import datetime, timezone

@app.route('/health')
def health():
    return jsonify({'status': 'healthy', 'timestamp':
datetime.now(timezone.utc).isoformat()})

@app.route('/health/details')
def health_details():
    checks = {
        'database': check_database(),
        'redis': check_redis(),
        'disk': check_disk_space()
    }
    status = 'healthy' if all(checks.values()) else 'unhealthy'
    return jsonify({'status': status, 'checks': checks})

@app.route('/metrics')
def metrics():
    return jsonify({
        'requests_total': get_request_count(),
        'active_connections': get_active_connections(),
        'memory_usage': get_memory_usage()
    })

def check_database():
    try:
        from sqlalchemy import text
        db.session.execute(text('SELECT 1'))
        return True
    except:
```

```
        return False

def check_redis():
    try:
        redis_client.ping()
        return True
    except:
        return False
```

十五、测试

```
import pytest
from app import create_app, db
from models import User

@pytest.fixture
def app():
    app = create_app('testing')
    with app.app_context():
        db.create_all()
        yield app
        db.drop_all()

@pytest.fixture
def client(app):
    return app.test_client()

@pytest.fixture
def auth_header(client):
    # 创建测试用户并获取token
    user = User(username='test', email='test@example.com')
    db.session.add(user)
    db.session.commit()

    resp = client.post('/login', json={
        'username': 'test',
        'password': 'password'
    })
    token = resp.get_json()['access_token']
    return {'Authorization': f'Bearer {token}'}

class TestUserAPI:
    def test_list_users(self, client):
        resp = client.get('/api/users')
        assert resp.status_code == 200
        assert isinstance(resp.get_json(), list)

    def test_create_user(self, client):
        resp = client.post('/api/users', json={
            'username': 'newuser',
```

```
            'email': 'new@example.com',
            'password': 'password123'
        })
        assert resp.status_code == 201
        assert resp.get_json()['username'] == 'newuser'

    def test_create_user_validation(self, client):
        resp = client.post('/api/users', json={'username': 'a'})
        assert resp.status_code == 400

    def test_protected_route(self, client, auth_header):
        resp = client.get('/api/protected', headers=auth_header)
        assert resp.status_code == 200

        resp = client.get('/api/protected')
        assert resp.status_code == 401

    def test_update_user(self, client, auth_header):
        resp = client.put('/api/users/1',
            headers=auth_header,
            json={'email': 'updated@example.com'}
        )
        assert resp.status_code == 200
        assert resp.get_json()['email'] == 'updated@example.com'

    def test_delete_user(self, client, auth_header):
        resp = client.delete('/api/users/1', headers=auth_header)
        assert resp.status_code == 204
```

十六、部署

1. Gunicorn

```
pip install gunicorn

# 命令行
gunicorn -w 4 -b 0.0.0.0:8000 --timeout 120 'app:create_app()'

# 配置文件 gunicorn.conf.py
bind = '0.0.0.0:8000'
workers = 4
worker_class = 'gevent'
timeout = 120
keepalive = 5
accesslog = '-'
errorlog = '-'
loglevel = 'info'
```

2. Docker

```dockerfile
FROM python:3.11-slim

WORKDIR /app

# 安装依赖
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# 复制代码
COPY . .

# 非root用户
RUN useradd -m appuser
USER appuser

EXPOSE 8000

CMD ["gunicorn", "-w", "4", "-b", "0.0.0.0:8000", "app:create_app()"]
```

docker-compose.yml

```yaml
version: '3.8'
services:
  web:
    build: .
    ports:
      - "8000:8000"
    environment:
      - DATABASE_URL=postgresql://user:pass@db:5432/app
      - REDIS_URL=redis://redis:6379/0
    depends_on:
      - db
      - redis

  db:
    image: postgres:15
    environment:
      POSTGRES_USER: user
      POSTGRES_PASSWORD: pass
      POSTGRES_DB: app
    volumes:
      - postgres_data:/var/lib/postgresql/data

  redis:
    image: redis:7

  celery:
    build: .
    command: celery -A tasks worker --loglevel=info
    depends_on:
```

```
      - redis

volumes:
  postgres_data:
```

3. Nginx

```
upstream flask_app {
    server 127.0.0.1:8000;
    server 127.0.0.1:8001;
    server 127.0.0.1:8002;
}

server {
    listen 80;
    server_name example.com;

    # HTTPS重定向
    return 301 https://$server_name$request_uri;
}

server {
    listen 443 ssl http2;
    server_name example.com;

    ssl_certificate /etc/nginx/ssl/cert.pem;
    ssl_certificate_key /etc/nginx/ssl/key.pem;

    # 安全头
    add_header Strict-Transport-Security "max-age=31536000;
includeSubDomains" always;
    add_header X-Frame-Options DENY always;
    add_header X-Content-Type-Options nosniff always;

    # API代理
    location / {
        proxy_pass http://flask_app;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_read_timeout 120s;
    }

    # WebSocket
    location /socket.io {
        proxy_pass http://flask_app;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection "upgrade";
    }
```

```
    # 静态文件
    location /static {
        alias /path/to/app/static;
        expires 30d;
    }

    # 限流
    limit_req_zone $binary_remote_addr zone=api:10m rate=10r/s;
    location /api/ {
        limit_req zone=api burst=20 nodelay;
        proxy_pass http://flask_app;
    }
}
```

十七、最佳实践

1. 应用工厂模式

```
# app/__init__.py
from flask import Flask, jsonify
from flask_sqlalchemy import SQLAlchemy
from flask_caching import Cache
from flask_limiter import Limiter

db = SQLAlchemy()
cache = Cache()
limiter = Limiter()

def create_app(config_name='default'):
    app = Flask(__name__)
    app.config.from_object(config[config_name])

    # 初始化扩展
    db.init_app(app)
    cache.init_app(app)
    limiter.init_app(app)

    # 注册蓝图
    from app.routes.api import api_bp
    from app.routes.auth import auth_bp

    app.register_blueprint(api_bp, url_prefix='/api')
    app.register_blueprint(auth_bp, url_prefix='/auth')

    # 注册错误处理
    register_error_handlers(app)

    return app
```

```
def register_error_handlers(app):
    @app.errorhandler(404)
    def not_found(error):
        return jsonify({'error': 'Not found'}), 404
```

2. 服务层模式

```
# services/user_service.py
from flask import abort
from werkzeug.security import generate_password_hash, check_password_hash

class UserService:
    @staticmethod
    def create(data):
        if User.query.filter_by(username=data['username']).first():
            raise ValueError('用户名已存在')

        user = User(
            username=data['username'],
            email=data['email'],
            password_hash=generate_password_hash(data['password'])
        )
        db.session.add(user)
        db.session.commit()
        return user

    @staticmethod
    def authenticate(username, password):
        user = User.query.filter_by(username=username).first()
        if user and check_password_hash(user.password_hash, password):
            return user
        return None

    @staticmethod
    def get_by_id(user_id):
        return db.session.get(User, user_id) or abort(404)

# routes/api.py
from flask import request, jsonify

@bp.route('/users', methods=['POST'])
def create():
    try:
        user = UserService.create(request.get_json())
    except ValueError as e:
        return jsonify({'error': str(e)}), 400
    return jsonify(user.to_dict()), 201
```

3. 安全实践

```
# 安全响应头
@app.after_request
def add_security_headers(response):
    response.headers['X-Content-Type-Options'] = 'nosniff'
    response.headers['X-Frame-Options'] = 'DENY'
    response.headers['X-XSS-Protection'] = '1; mode=block'
    response.headers['Strict-Transport-Security'] = 'max-age=31536000;
includeSubDomains'
    return response

# 输入验证（使用ORM避免SQL注入）
# 正确
user = User.query.filter_by(username=username).first()
# 错误
# db.execute(f"SELECT * FROM users WHERE username = '{username}'")

# XSS防护（Jinja2自动转义）
# 在模板中使用 {{ variable }} 会自动转义

# CSRF保护
from flask_wtf.csrf import CSRFProtect
csrf = CSRFProtect(app)

# API豁免CSRF（在蓝图注册时设置）
# api_bp = Blueprint('api', __name__, url_prefix='/api')
# csrf.exempt(api_bp)  # 豁免整个蓝图
```

4. 日志规范

```
import logging
import time
from logging.handlers import RotatingFileHandler
from flask import request, g
import structlog

# 基本配置
def setup_logging(app):
    if not app.debug:
        handler = RotatingFileHandler(
            'app.log',
            maxBytes=10*1024*1024,
            backupCount=5
        )
        handler.setLevel(logging.INFO)
        handler.setFormatter(logging.Formatter(
            '%(asctime)s - %(name)s - %(levelname)s - %(message)s'
        ))
        app.logger.addHandler(handler)

# 结构化日志
```

```
structlog.configure(
    processors=[
        structlog.processors.TimeStamper(fmt="iso"),
        structlog.processors.JSONRenderer()
    ]
)
logger = structlog.get_logger()

# 请求日志
@app.before_request
def log_request():
    logger.info("request_started",
        method=request.method,
        path=request.path,
        ip=request.remote_addr
    )

@app.after_request
def log_response(response):
    logger.info("request_completed",
        status=response.status_code,
        duration=time.time() - g.start_time
    )
    return response
```

5. 性能优化

```
# 数据库查询优化
# 只查询需要的字段
users = User.query.with_entities(User.id, User.username).all()

# 延迟加载
users = User.query.options(db.defer('large_content')).all()

# 预加载关联
users = User.query.options(db.joinedload('posts')).all()

# 批量操作
db.session.bulk_insert_mappings(User, [
    {'username': 'user1', 'email': 'user1@example.com'},
    {'username': 'user2', 'email': 'user2@example.com'},
])

# 连接池
app.config['SQLALCHEMY_ENGINE_OPTIONS'] = {
    'pool_size': 10,
    'max_overflow': 20,
    'pool_timeout': 30,
    'pool_recycle': 3600
}
```

```
# 响应压缩
from flask_compress import Compress
Compress(app)
```

6. API文档

```
pip install flasgger
```

```
# 使用Flasgger
from flasgger import Swagger, swag_from
from flask import jsonify

swagger = Swagger(app)

@app.route('/api/users')
@swag_from({
    'responses': {
        200: {
            'description': '用户列表',
            'schema': {
                'type': 'array',
                'items': {
                    'properties': {
                        'id': {'type': 'integer'},
                        'username': {'type': 'string'}
                    }
                }
            }
        }
    }
})
def list_users():
    return jsonify([u.to_dict() for u in User.query.all()])

# 或使用装饰器
@app.route('/api/users/<int:user_id>')
@swag_from('docs/get_user.yml')
def get_user(user_id):
    pass
```

docs/get_user.yml

```
parameters:
  - name: user_id
    in: path
    type: integer
    required: true
```

```
      description: 用户ID
  responses:
    200:
      description: 用户信息
      schema:
        properties:
          id:
            type: integer
          username:
            type: string
          email:
            type: string
    404:
      description: 用户不存在
```